Project - High Level Design

on

Multi-Agent Education System

Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	AMAN VERMA	EN22CS301110
02	ANJALI JAISWAL	EN22CS301132
03	ANKIT PARMAR	EN22CS301135
04	ANSH LAAD	EN22CS301141
05	APOORVA GUPTA	EN22CS301190
06	ARYA BHAWSAR	EN22CS301209

*Group Name:*12D2

*Project Number:*AAI-24

Industry Mentor Name:

University Mentor Name:Ajeet Singh Rajput

Academic Year:2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The **Multi-Agent Education System** is an intelligent application built using **Agentic AI architecture**, where multiple specialized agents collaborate to generate high-quality educational content. Traditional AI systems typically rely on a single model to process user queries, which may result in unstructured or incomplete responses. To address this limitation, this project introduces a **multi-agent collaborative framework** using CrewAI that enables task delegation, structured reasoning, and multimodal output generation.

The system includes multiple agents such as an **Orchestrator Agent**, **Content Generation Agent**, and **Image Generation Agent** that communicate through defined workflows. Each agent performs a specific function, improving the quality, modularity, and scalability of the system. The final output consists of well-structured educational explanations along with relevant images to enhance conceptual understanding.

1.1 Scope of the Document

This document describes the architecture and design of the **Multi-Agent Education System**. It explains how multiple AI agents collaborate to process educational queries and generate structured outputs.

The document includes:

- System architecture overview
- Description of agents and their responsibilities
- Workflow and process design
- Information exchange between agents
- API usage and integration
- Design considerations and scalability aspects

The scope of this document is limited to the conceptual design and implementation structure of the multi-agent educational system.

1.2 Intended Audience

This document is intended for:

*__Students and learners__ studying Agentic AI and multi-agent systems
*__Developers and engineers__ who want to implement AI-based educational systems
*__Project evaluators and instructors__ assessing the system design and implementation
*__Researchers__ interested in collaborative AI and intelligent agent architectures

1.3 System Overview

The Multi-Agent Education System consists of multiple AI agents working collaboratively under the control of an orchestrator.

Key Components:

Orchestrator (Crew Manager): Controls workflow and manages communication between agents

Researcher Agent: Collects relevant study material based on user input

Writer Agent: Converts raw data into structured and readable educational content

LLM API: Provides intelligence for generating responses

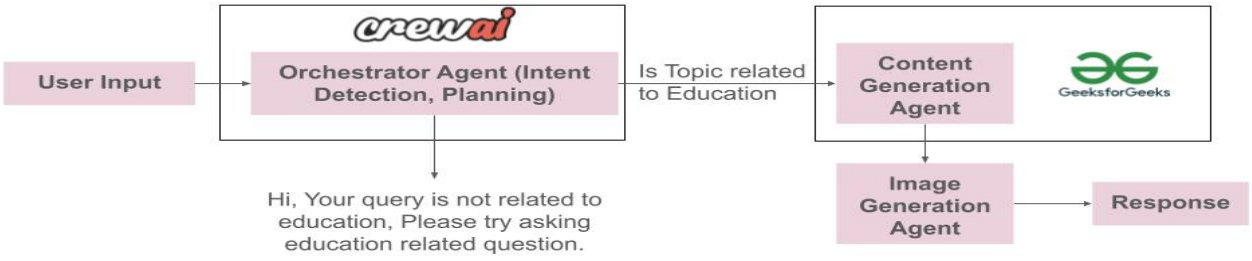
Image Generation Agent: Produces diagrams or visual learning aids.

Working Principle

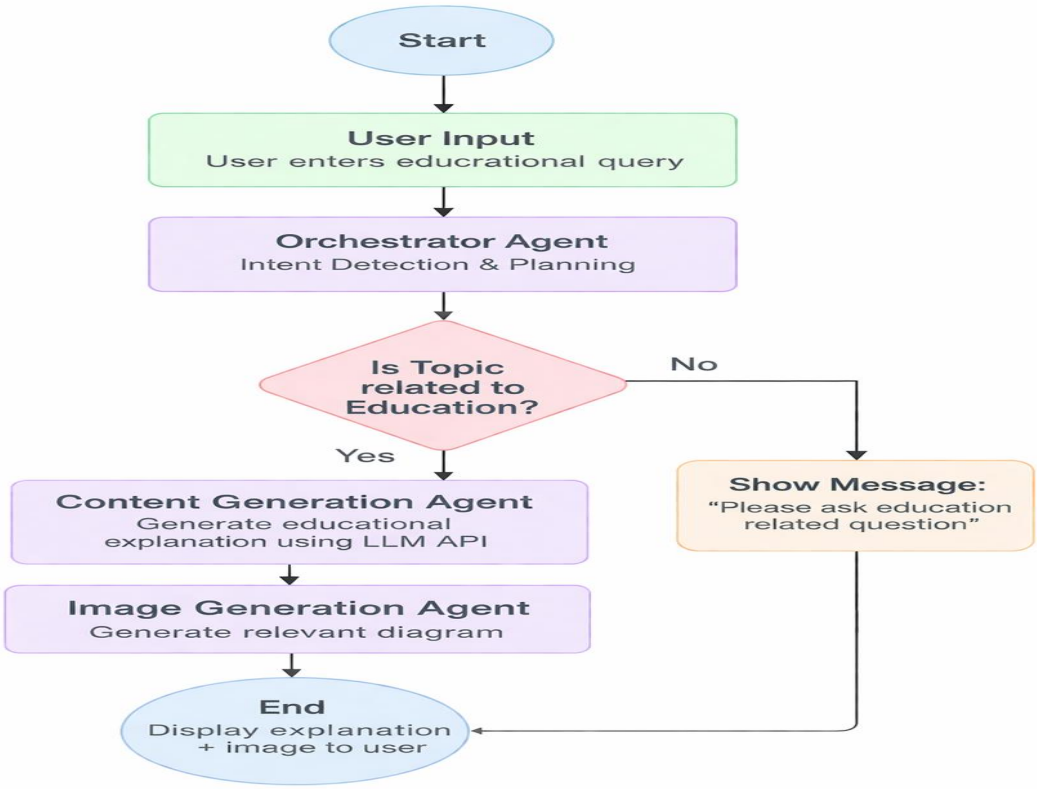
- User provides an educational query or topic through the interface.
- The Orchestrator Agent analyzes the query and checks whether it is related to education.
- If the query is not related to education, the system returns a message asking the user to enter an education-related question.
- If the query is valid, the Orchestrator Agent assigns the task to the Content Generation Agent.
- The Content Generation Agent generates structured educational content using the LLM API.
- The Image Generation Agent generates a relevant diagram or visual representation of the topic.
- The final response, including both explanation and image, is displayed to the user.

2.System Design

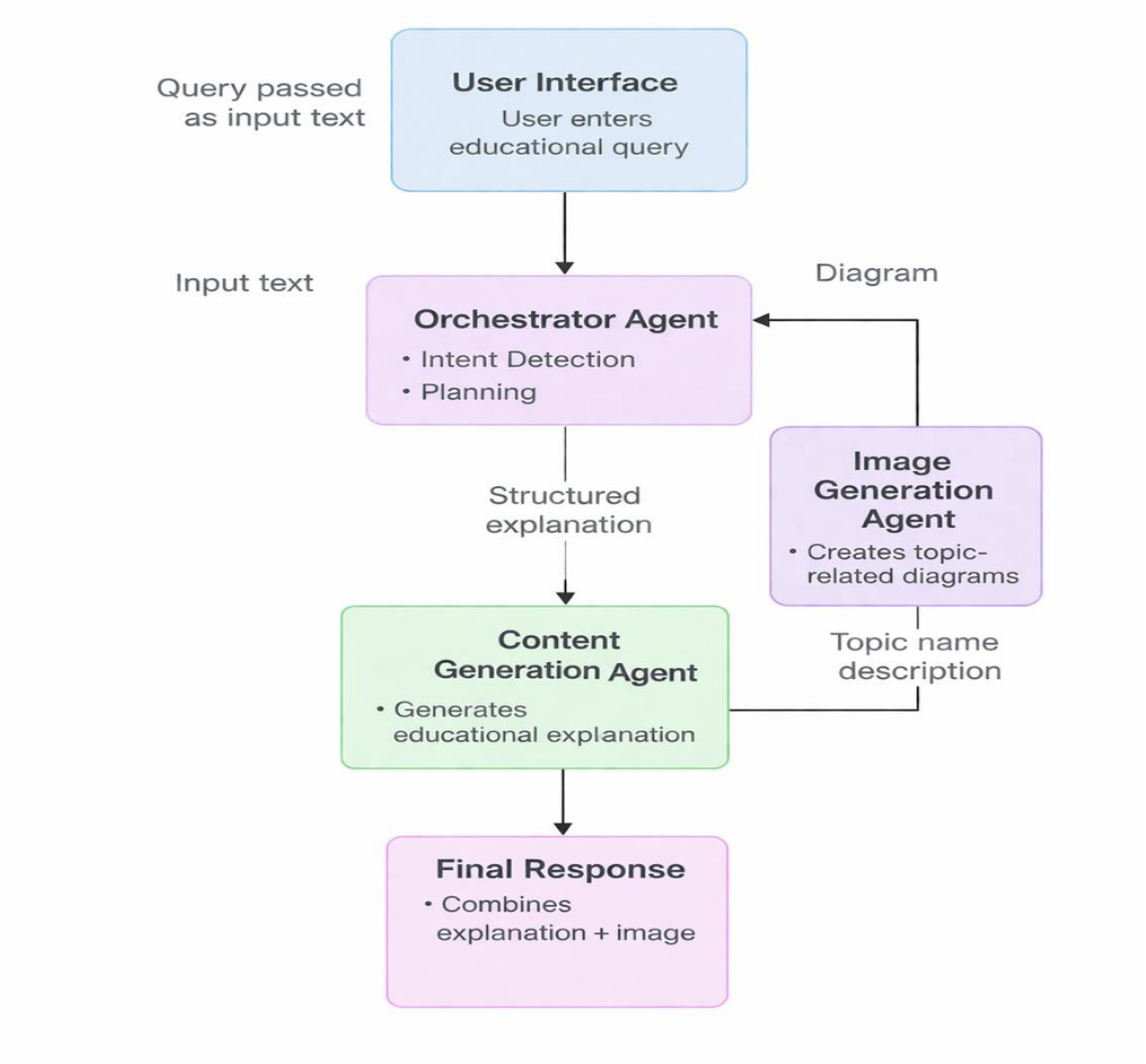
2.1 Application Design



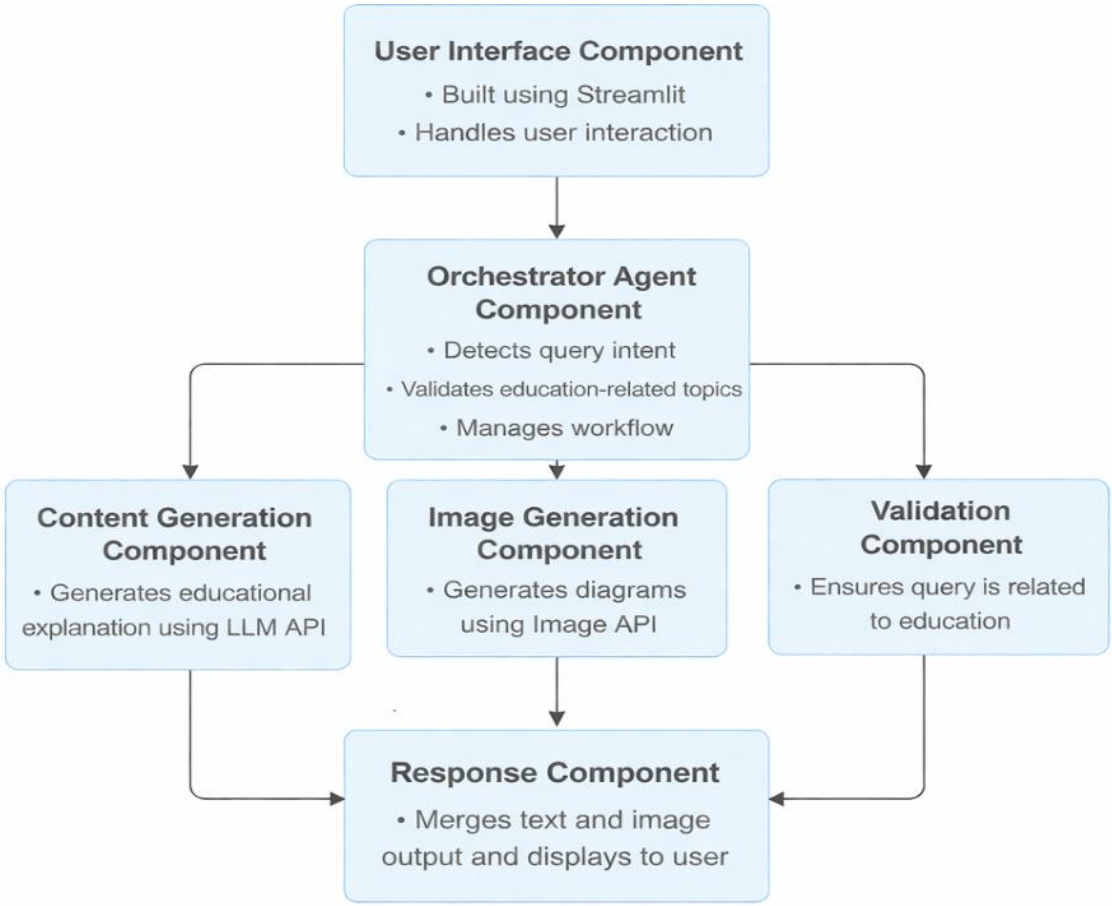
2.2 Process Flow



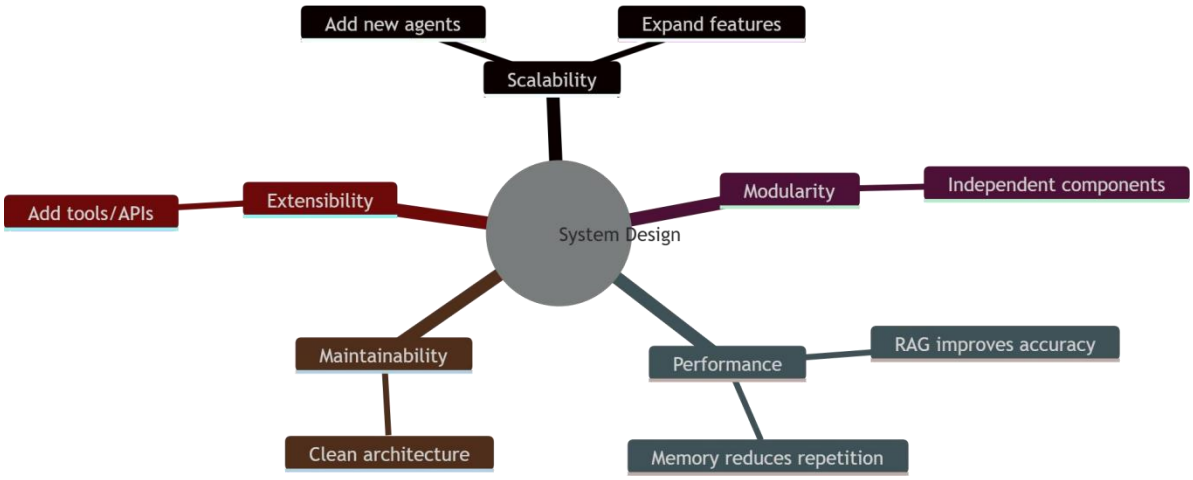
2.3 Information Flow



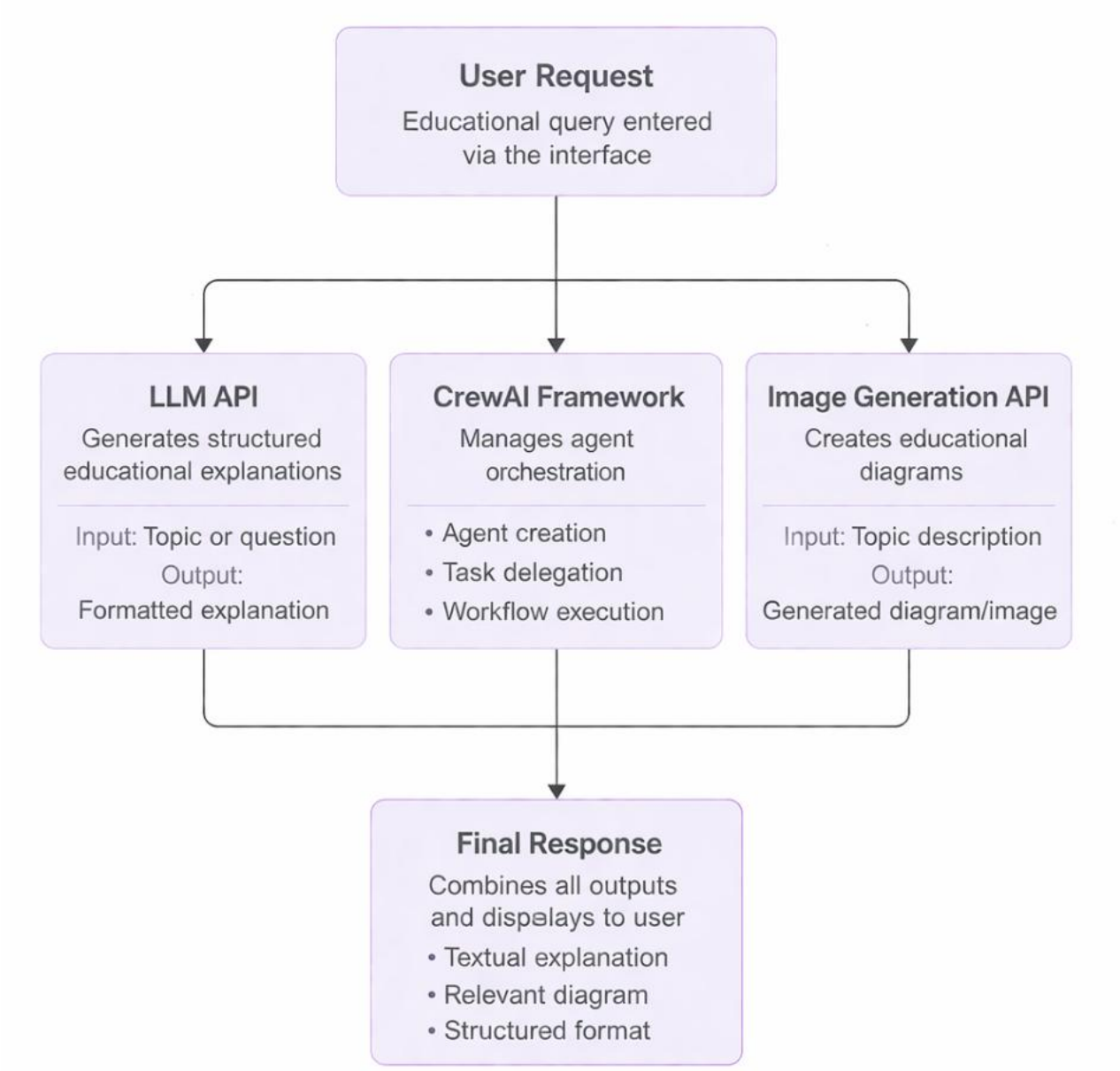
2.4 Components Design



2.5 Key Design Considerations



2.6 API Catalogue.



3. Data Design(Stateless System)

The Data Design of the **Multi-Agent Education System** defines how data is structured, processed, stored, and maintained across different agents. Since the system relies on multiple agents collaborating in a workflow, efficient data handling is essential to ensure smooth communication, scalability, and high-quality output generation.
The system primarily handles **user queries, agent outputs, workflow states, and generated responses**, using structured formats for seamless inter-agent communication.

3.1 Data Model

The system does **not maintain a traditional data model** because it does not store or manage persistent data.
Instead, it follows a **dynamic, request-based data model** where:

- Input is received from the user
- Data is processed temporarily by agents
- Output is generated in real-time using the LLM API

Transient Data Components

1. **User Input**
   - Topic or question entered by the user
2. **Intermediate Agent Data**
   - Intent detected by Orchestrator
   - Processed content by Content Agent
   - Image prompt for Image Agent
3. **Final Output**
   - Generated explanation (text)
   - Generated image (if applicable)

* All data exists **only during execution** and is discarded after response generation.

3.2 Data Access Mechanism

Since no database is used, the system relies entirely on **in-memory and API-based data access**.

Mechanism Flow

1. User provides input
2. Orchestrator processes the query
3. Data is passed between agents internally (in memory)
4. Content Agent calls LLM API

5. User provides input
6. Orchestrator processes the query
7. Data is passed between agents internally (in memory)
8. Content Agent calls LLM API
9. Image Agent generates visuals
10. Final response is returned

Key Characteristics

- No database queries
- No file storage
- No persistent memory
- Fast execution using **temporary variables**

3.3 Data Retention Policies

The system follows a **zero-retention policy**.

Policy Details

- No user data is stored
- No conversation history is maintained
- No logs are persisted permanently

Session Behavior

- Data exists only during runtime
- Automatically cleared after response

Advantages

- High privacy
- No data leakage risk
- Simple system design

3.4 Data Migration

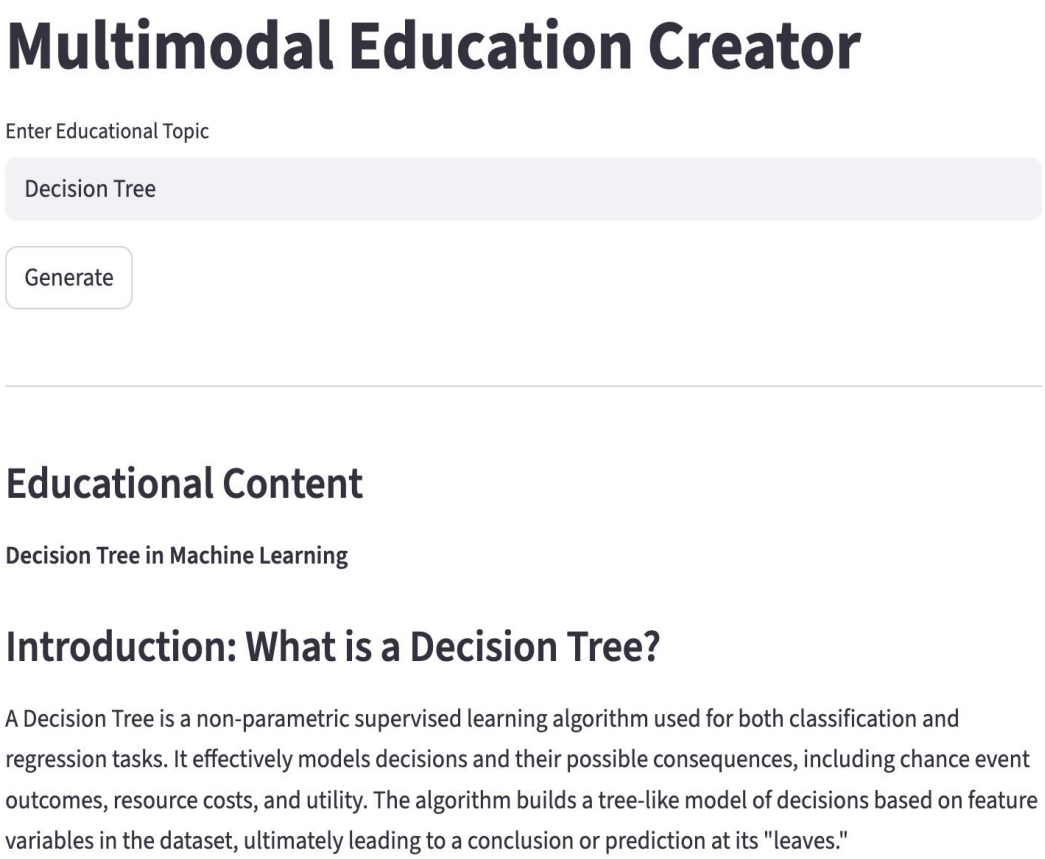
Data migration is **not applicable** in this system because:

- No database is used
- No persistent storage exists
- No historical data is maintained

Exception (Future Scope)
If storage is added later:

- Migration tools and schema handling may be required
- Data backup and transfer strategies can be introduced

4 Interfaces



At its core, a Decision Tree is a flowchart-like structure. Each internal node within this structure represents a "test" on an attribute (for example, "Is the outlook sunny?"). Each branch stemming from an internal node represents the outcome of that test, and each leaf node represents a final class label (in classification tasks) or a numerical value (in regression tasks), or a decision. The paths traced from the root of the tree down to a leaf node collectively represent classification rules.

The fundamental idea behind decision trees is to recursively divide the data into smaller, more manageable subsets. This division is based on the values of the input features, continuing until each subset contains data points that are as homogeneous as possible with respect to the output variable (the target).

2. How it Works (Intuition)

To grasp the intuition behind Decision Trees, imagine you're trying to decide whether to play tennis. Your decision process likely involves considering several factors, such as the weather outlook, temperature, humidity, and wind. A decision tree mimics this human decision-making process by following a similar sequential approach:

- **Step 1:** You first look at the most important factor (e.g., Outlook).
- **Step 2:** If the Outlook is Rainy, you then consider another factor (e.g., Wind).
- **Step 3:** Based on the Wind, you make a final decision (e.g., if Windy, don't play; if Not Windy, play).

The Decision Tree algorithm learns these "rules" and the order of importance of factors directly from historical data.

3. Key Terminology

Understanding the specific terminology is crucial for comprehending Decision Trees:

- **Root Node:** This is the topmost node in the tree. It represents the entire dataset before any splits occur and is the starting point for all decisions.
- **Internal Node (Decision Node):** Any node that has two or more branches extending from it. It represents a test or condition on an attribute, and each branch from it represents one of the possible outcomes of that test.
- **Leaf Node (Terminal Node):** A node that does not split further. It signifies the final decision or prediction, such as a class label in classification or a numerical value in regression.
- **Splitting:** The process of dividing a node into two or more sub-nodes based on a chosen feature and a specific split criterion.
- **Branch/Sub-tree:** A section of the entire tree, representing a specific decision path or a smaller tree structure within the main tree.
- **Parent Node:** A node that is divided into one or more sub-nodes.
- **Child Node:** The sub-nodes that branch out directly from a parent node.
- **Pruning:** The process of removing sub-nodes of a decision node. It is the opposite of splitting and is primarily used to reduce the complexity of the tree and prevent overfitting to the training data.

4. Splitting Criteria (Attribute Selection Measures)

To construct the tree, at each internal node, the algorithm must decide which attribute to split on. This decision is made using various metrics, known as Attribute Selection Measures (ASM), which evaluate the "purity" or "homogeneity" of a node. The primary objective is to select the split that results in the most homogeneous (or "pure") child nodes, meaning data points within each child node are very similar with respect to the target variable.

The most common criteria include:

- **Gini Impurity (for Classification):**
  - Measures how often a randomly chosen element from the set would be incorrectly labeled if it were randomly labeled according to the distribution of labels in the subset.
  - A Gini impurity of 0 means all elements belong to a single class (perfectly pure node). A Gini impurity of 0.5 (for a binary classification) indicates maximum impurity, meaning elements are equally distributed across classes.
  - The algorithm chooses the split that minimizes the weighted average Gini impurity of the child nodes.
  - Formula for a node *t*: $Gini(t) = 1 - \sum_{i=1}^{c}(P_i)^2$, where P_i is the probability of class *i* at node *t*, and *c* is the number of classes.

5. Algorithm Steps (Simplified)

The process of building a Decision Tree can be broken down into these recursive steps:

1. **Start at the Root Node:** Begin the tree construction process with the entire training dataset at the root node.
2. **Select the Best Attribute:** Use an Attribute Selection Measure (such as Gini impurity, Information Gain, or Variance Reduction) to evaluate all available attributes and find the one that offers the best split. The "best" attribute is defined as the one that creates the most homogeneous child nodes.
3. **Create Child Nodes:** Split the dataset into two or more subsets based on the distinct values or ranges of the chosen "best" attribute. Each subset forms a new child node.
4. **Recursion:** Repeat steps 2 and 3 for each newly created child node. This recursive process continues until certain stopping conditions are met.
5. **Stopping Condition:** The splitting process halts when one or more of the following conditions are met:
   - All data points within a node belong to the same class (indicating a perfectly pure node).
   - There are no remaining attributes to split on, or all remaining attributes have already been used.
   - A pre-defined stopping criterion is met, such as reaching a maximum tree depth, requiring a minimum number of samples to split a node, or having a minimum number of samples in a leaf node.
6. **Assign Leaf Nodes:** Once a stopping condition is met for a particular branch, that node becomes a leaf node. Its label or value is assigned based on the majority class of the samples within that node (for classification) or the average value of the samples (for regression).

6. Pros of Decision Trees

Decision Trees offer several compelling advantages:

- **Simple to Understand and Interpret:** The tree structure is highly intuitive and can be easily visualized. This makes the decision-making logic transparent and comprehensible, even for non-technical users.
- **Minimal Data Preparation:** They require less data cleaning and preprocessing compared to some other algorithms. Decision trees can handle both numerical and categorical data without extensive feature scaling or normalization.
- **Non-parametric:** They make no assumptions about the underlying distribution of the data, which makes them flexible for various datasets.
- **Handles Multi-output Problems:** Decision trees can be extended to predict multiple output variables simultaneously.
- **Implicit Feature Selection:** Features that are more important for classification or regression naturally appear higher up in the tree, effectively performing an implicit form of feature selection.
- **Basis for Ensemble Methods:** They serve as the fundamental building blocks for more powerful and robust ensemble learning techniques like Random Forests and Gradient Boosting.

7. Cons of Decision Trees

Despite their advantages, Decision Trees also have limitations:

- **Prone to Overfitting:** Without proper regularization or pruning, a decision tree can create overly complex trees that fit the training data too closely. This can lead to poor generalization performance on unseen data.
- **Instability:** Small variations in the training data can sometimes result in a completely different tree structure. This sensitivity arises because the greedy approach to splitting can lead to different initial splits influencing subsequent branches significantly.
- **Local Optima:** The greedy algorithm used to build decision trees selects the best split at each step without looking ahead. This means it may converge to a locally optimal tree rather than a globally optimal one, potentially missing a better overall tree structure.
- **Bias Towards Dominant Classes:** If the dataset contains imbalanced classes, decision trees can be biased towards the majority class. It is often recommended to balance the dataset prior to fitting a decision tree or use techniques like class weighting.
- **Complexity for Continuous Variables:** Splitting on continuous variables can be computationally intensive as it requires evaluating many possible split points to find the optimal threshold.

8. Use Cases

Decision Trees are versatile and applied in various domains, including:

- **Medical Diagnosis:** Assisting in identifying diseases based on patient symptoms and medical test results.
- **Customer Churn Prediction:** Predicting which customers are most likely to discontinue a service or subscription.
- **Credit Risk Assessment:** Evaluating the creditworthiness of loan applicants based on their financial history and other attributes.
- **Fraud Detection:** Identifying potentially fraudulent transactions in financial or insurance industries.
- **Predicting Product Demand:** Estimating future sales for specific products based on historical data and market trends.

9. Example (Conceptual)

Let's illustrate a simplified Decision Tree to decide if a person will buy a product based on their Age, Income, and Education level:

```
        [Root Node]
           Age <= 30?
          /         \
        Yes         No
       /             \
    [Node 1]          [Node 2]
    Income <= 50k?    Education = High?
    /      \          /          \
  Yes     No         Yes         No
  /        \        /           \
 Buy     Don't Buy  Don't Buy   Buy
 (Leaf)  (Leaf)     (Leaf)      (Leaf)
```

In this conceptual tree:

- The process starts at the **Root Node** by evaluating the 'Age' attribute.

- If 'Age' is less than or equal to 30, the path leads to **Node 1**, which then checks 'Income'.
- If 'Income' is less than or equal to 50k, the prediction is 'Buy' (a **Leaf Node**).
- Conversely, if 'Age' is greater than 30, the path goes to **Node 2**, checking 'Education'.
- If 'Education' is 'High', the prediction is 'Don't Buy' (another **Leaf Node**).

10. Relationship to Ensemble Methods

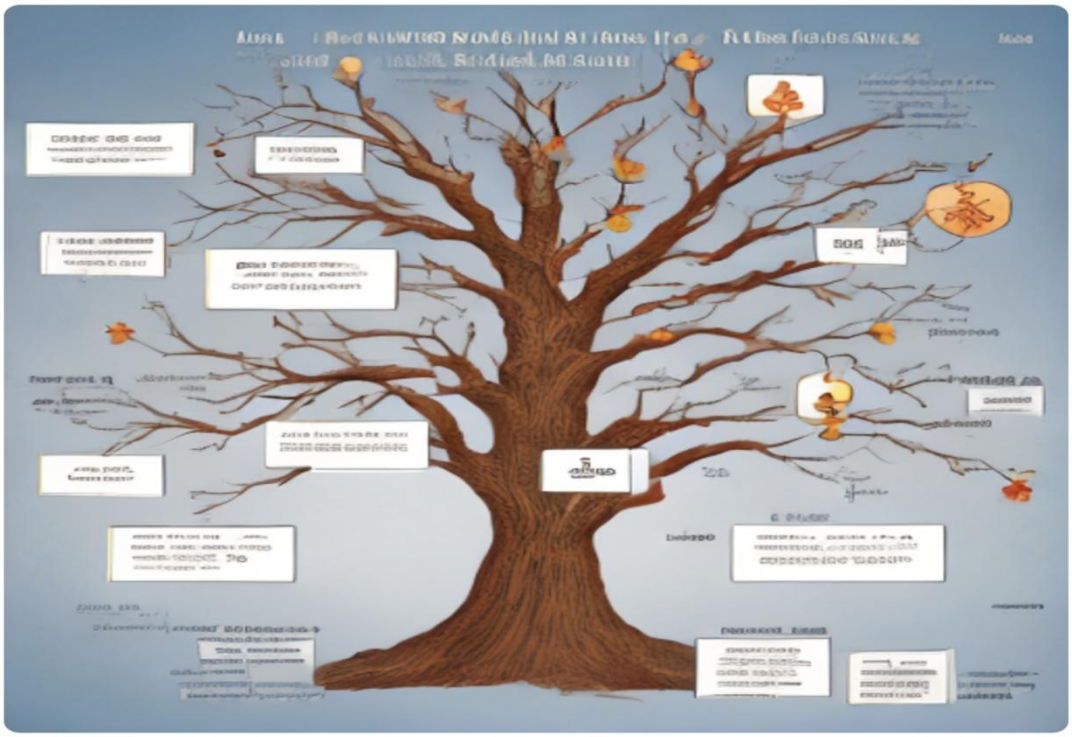
Decision Trees serve as the foundational components for some of the most powerful and widely used ensemble learning methods:

- **Random Forests:** This ensemble technique constructs multiple decision trees. Each tree is trained on a random subset of the data (bootstrapping) and considers only a random subset of features at each split. The final prediction is derived by averaging the predictions of all individual trees (for regression) or by taking a majority vote (for classification). This collective approach significantly reduces overfitting and improves the model's robustness and accuracy.
- **Gradient Boosting (e.g., XGBoost, LightGBM):** In contrast to Random Forests, Gradient Boosting builds trees sequentially. Each new tree in the sequence is trained to correct the errors (or residuals) made by the combination of all previously built trees. It iteratively focuses on the instances that were misclassified or had larger residuals, gradually improving the model's overall accuracy and performance.

Conclusion

Decision Trees are fundamental and intuitive machine learning algorithms that provide a clear, interpretable way to make predictions. They are non-parametric supervised learning models used for both classification and regression, characterized by their flowchart-like structure that guides decisions from a root node to leaf nodes through sequential splits based on feature attributes. While they are easy to understand, require minimal data preparation, and offer implicit feature selection, they are also prone to overfitting and instability if not properly managed through techniques like pruning. Despite these limitations, their simplicity and effectiveness make them a crucial component and a foundational building block for more powerful ensemble techniques like Random Forests and Gradient Boosting, which significantly enhance their performance and robustness.

Educational Diagram



5. State and Session Management

The system follows a **stateless architecture**, meaning no long-term session or state is maintained.

State Handling

- Each request is processed independently
- No previous interaction is stored
- Agents operate only on the current input

Session Management

- Temporary session exists only during execution
- Data flow:
  - *User Input → Orchestrator → Agents → Response*
- Session ends immediately after response generation

Transient State Components

- Query context (temporary)
- Agent communication data
- LLM-generated intermediate outputs

*All session data is **discarded after execution**

6. Caching

The system currently **does not use caching** due to real-time generation requirements.

Reason for No Caching

- Responses are dynamically generated
- Each query may produce unique outputs
- Avoids stale or outdated information

Future Scope (Optional)

Caching can be introduced for:

- Frequently asked questions (FAQs)
- Repeated queries

Possible Caching Strategies

- In-memory caching (Redis)
- API response caching

- Prompt-result caching

7. Non-Functional Requirements

7.1 Security Aspects

Input Validation

- Ensure queries are relevant to education
- Prevent malicious or invalid inputs

API Security

- Secure API keys for LLM services
- Use environment variables for credentials

Data Privacy

- No user data is stored
- Zero-retention ensures high privacy

Access Control

- Restrict unauthorized API usage
- Implement rate limiting

7.2 Performance Aspects

Response Time

- Fast processing using in-memory execution
- Dependent on LLM API latency

Scalability

- Easily scalable due to stateless design
- Can handle multiple requests in parallel

Reliability

- System depends on API availability

- Retry mechanisms can be added

Efficiency

- No database overhead
- Lightweight architecture

8. References

This project is based on concepts and technologies from the fields of **Agentic AI**, **Large Language Models (LLMs)**. Frameworks such as CrewAI and LangChain were used to design and manage the multi-agent workflow, while Python served as the primary implementation language. The system integrates LLM APIs to generate intelligent responses. Additional references include official documentation, research materials, and online resources related to multi-agent systems, AI orchestration, and modern NLP techniques, which collectively supported the design and development of this system.

Research Papers & Concepts

1. Russell, S., & Norvig, P.
   Artificial Intelligence: A Modern Approach
   Pearson Education
   Fundamental concepts of intelligent agents and AI systems.
2. OpenAI
   GPT Models and Generative AI Documentation
   Concepts of Large Language Models and text generation.
3. Lewis, P., et al.
   Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
   (2020)
   Introduction to RAG architecture.